

Weekend Deployment Challenge: Morcegao

#deployment

What the App Does

Morcegao is a Discord bot built to keep a community informed without making moderators repeat the same announcements by hand. From a member's perspective, the bot offers slash commands such as `/ping`, `/commands`, `/welcome`, `/resend_live`, and `/clear`. It can announce when a configured Twitch channel goes live, check for new Instagram posts through Meta's official Graph API, create temporary voice rooms, and add VAMPI letter reactions to images, GIFs, and videos posted in a configured media channel.

The app solves a small but real community operations problem: information is easy to miss when it is scattered across platforms. Twitch activity, Instagram posts, and Discord moderation tasks now meet in the same place. Each integration is independently enabled through environment variables, so the Discord bot remains useful even if Twitch or Instagram is intentionally disabled. Secrets are read from environment configuration and are never written to source code or logs.

How I Built It

I built Morcegao with Python 3.12 and discord.py 2.x. The main client extends commands.Bot and uses an application command tree for slash commands. Features are organized as cogs, which keeps general commands, temporary voice channels, and media reactions isolated while still using one connected Discord client. The code uses async and await throughout, allowing the Discord gateway, background tasks, and HTTP webhook server to run together without blocking one another.

One key decision was to keep the FastAPI webhook server inside the same process as the Discord bot. The application starts FastAPI on localhost only, then Nginx proxies the public Twitch webhook route to it. This avoids a second Discord client and ensures that the webhook delivers events to the same in-memory notification queue used by the bot. Twitch EventSub signatures are validated before events are processed. For Instagram, I chose polling through the official Meta API rather than scraping, browser automation, or unofficial APIs.

The most practical deployment challenge was turning a local Python process into a service that survives a closed SSH session and a server restart. I used a Python virtual environment, a dedicated non-admin Linux account, and a systemd service configured to restart after failure. Unit tests use fakes and mocks, so the project can validate Discord, Twitch, Instagram, and webhook behavior without real calls during development. Ruff and type hints provide another safety net before deploys.

AWS Services Used and Architecture Overview

The live deployment runs on Amazon EC2 with Amazon Linux 2023. EC2 provides the persistent compute environment for Python, discord.py, FastAPI, Nginx, systemd, and the local SQLite notification store. The VPC security group limits inbound access: SSH is restricted to the administrator, while ports 80 and 443 are used for TLS validation and the public Twitch callback. FastAPI remains bound to 127.0.0.1:8000 and is never exposed directly to the internet.

Nginx is the HTTPS reverse proxy in front of the local FastAPI server. It forwards only the Twitch webhook path, while Let's Encrypt supplies a renewable TLS certificate. A DuckDNS hostname gives the EC2 instance a public callback address. A future security improvement is already prepared in the configuration: Instagram token storage can move from a local environment file to AWS Secrets Manager with an EC2 IAM role and least-privilege GetSecretValue and PutSecretValue permissions.

Twitch EventSub	Nginx HTTPS	FastAPI localhost	Discord Bot
Instagram Graph API			Discord Community

What I Learned

This deployment made the difference between running code locally and operating an application continuously much clearer. I learned to expose only the network paths that the app actually needs, keep internal services on localhost, and use Nginx to separate public HTTPS traffic from the application process. I also learned that a systemd service is not just a convenience: it gives a small application predictable startup, logging, recovery, and maintenance commands.

I also gained experience combining asynchronous Discord bot work with an HTTP webhook in one event loop. Slash command synchronization in a development guild made iteration faster, while configuration validation kept malformed IDs and missing secrets from producing confusing runtime failures. The project reinforced a simple lesson: a focused app can be valuable when it has clear boundaries, secure defaults, useful automation, and deployment documentation that someone else can follow.

Live App Link

The deployed health endpoint is available at <https://morcegao-bot.duckdns.org/health>. It demonstrates that the EC2 instance, Nginx reverse proxy, TLS certificate, and FastAPI service are live. The Discord bot is intentionally interacted with inside its Discord server, where members can use its slash commands and receive notifications.